ACADEMIA
AI & APPLICATIONS

Research Article
Published: 31 March 2026
https://doi.org/10.20935/AcadAI8228

Using large language models to generate and correct code written in low-resource and domain-specific programming languages

Manuel Alfonseca[1,*], Arturo Martín Colino[2]

Academic Editors: Nie Weizhi, Zenglin Xu

Abstract

Introduction: Large language models (LLMs) are being used increasingly by programmers to generate code. Apparently, their use seems to be reasonably acceptable when languages such as Python are used, because the Internet contains huge amounts of information and coding examples written in those highly used languages, and this information has been used to train LLMs. On the other hand, little-used languages have much lower resources, while domain-specific languages incorporate features difficult to interpret and incorporate by code-generating tools.

Materials and methods: In this article, we selected IBM APL2 as a low-resource language and EES as a domain-specific language and analyzed this situation by requesting four highly used LLMs to generate code (which came out riddled with errors) and to correct the code generated by the same and the other LLMs, with appalling results.

Results: As could be foreseen, the LLM were not successful. Several suggestions are provided to improve the situation, especially in programming education, to prevent the loss of diversity in the selection of programming languages. The limitations of our approach have also been discussed.

Conclusions: The experiments described in this article demonstrate that LLMs do not currently generate useful code, when requested to use low-resource and domain-specific programming languages.

Keywords: *large language models, automatic programming, automatic program debugging, programmer education*

Citation: Alfonseca M, Martín Colino A. Using large language models to generate and correct code written in low-resource and domain-specific programming languages. *Academia AI and Applications* 2026;2. https://doi.org/10.20935/AcadAI8228

1. Introduction

Computer programming is a tedious activity, and the possibility of its automation has been considered desirable since the beginning of electronic computers, over 75 years ago. In fact, attempts in this direction are quite old. The first method of some kind of automation (albeit incipient) was attempted was the development of program libraries that could be used by programmers to simplify their efforts.

During the eighties, a new way of programming appeared, object-oriented programming [1], one of whose objectives was to increase the reuse of previous programming efforts, which would be incorporated in *methods* associated with *object classes*. To some extent, this was an extension of the program libraries associated with previous computer languages, such as FORTRAN, COBOL, or PASCAL.

The nineties saw an important development of research in genetic algorithms and evolutionary computation, which gave rise to evolutionary programming, whose objective was the automatic generation of complete programs by means of grammar evolution and other branches of this field of computer science. References [2–5] describe the fundamentals and provide a few examples of this technique.

The last few years have witnessed a revolutionary advance in the field of automatic computer programming with the appearance of large language models (LLMs), which generate text as a function of their training by means of hundreds of thousand files distributed on the Internet [6, 7]. The first LLM to get the attention of the public was ChatGPT (OpenAI, 2022), soon followed by similar applications, such as Claude (Anthropic, 2023), GEMINI (Google, 2024) and DeepSeek (DeepSeek, 2025).

LLMs can also be trained by means of a large number of computer programs obtained from platforms such as GitHub and similar, in such a way that the same algorithm that generates texts similar to those written by human beings can also be used to do automatic programming, automatic error fixing, automatic code interpretation [8], and automatic code evaluation [9]. A few of them are even more ambitious and try to incorporate the new techniques in Integrated Development Environments (IDEs), which intend to offer design alternatives to the programmers [10, 11].

Several recent publications review the use of LLMs to automatically generate code [7, 12–14]. These reviews have pointed out possible problems in the use of LLMs for automatic programming, as the code they generate is usually loaded with errors

[1]Departament of Computer Engineering, Higher Politechnic School, Autonomous University of Madrid, Madrid, Spain.
[2]School of Engineering (ICAI), Comillas Pontifical University, Madrid, Spain.
*email: manuel.alfonseca@uam.es

(*hallucinations*, in the LLM parlance). Reference [12], for instance, addresses the problem by using agents that invoke the LLM and correct the errors.

The following is one conclusion of reference [9]: "*[C]ode generation techniques can successfully handle more complex tasks than before... However, research around the evaluation of LLM generated code fails to keep pace with the application... [W]e think more effort is needed to fill the gap in the future*". This is especially important in the case of low-resource programming languages (LRPLs) and domain-specific languages (DSLs) (LRPLs can be defined as those languages with little data available, while DSLs are languages designed and optimized for specific applications [14]), as indicated in [14], a review that is focused precisely on those two families of languages, although the set of languages considered is not exhaustive.

Some of these studies [15] come to the conclusion that using LLMs to generate code with the same information used to generate text is not the way to go. They think that it would be better to train specialized LLMs with preprogrammed code libraries, written by programmers, which would be devoid of errors and therefore would be much more efficient.

References such as [16] focus on analyzing errors in programs generated by LLMs, written in highly used languages, like Python. The authors of this reference appear to believe that syntax errors are not common in code generated by LLMs, while most mistakes have to do with reasoning, management of many conditions, or just efficiency. We have found that, on the contrary, syntactic errors are quite common in generated code written in LRPL and DSL, as described in Section 3 of this paper.

One important application of automatic programming is education. The use of LLMs can help programming students to handle tasks of program design and programming, program interpretation and program debugging. However, not all the experiences are easy and positive, as indicated in [17].

This paper addresses the problem of code generation and code debugging in the case of two languages, one of them (IBM APL2) belonging to the LRPL category and the other (EES) to the DSL category. These languages are described in Section 2. Section 3 describes the experiments performed in this direction using the four LLMs mentioned above. In Section 4 we offer a few suggestions with respect to the use of LLMs for code generation and for the education of programmers.

2. Materials and methods

As most studies performed on LLM-helped code generation address widely used computer languages (especially Python), we decided to tackle two different families of languages:

- Low-resource programming languages (LRPLs). We chose IBM APL2.
- Domain-specific languages (DSLs). We selected EES (V12.085).

In both cases, this paper describes two different approaches:

- Asking the LLMs to generate code in the selected language/dialect.
- Asking the LLMs to correct the code generated by the other LLMs, or by themselves.

The four LLMs we tested were chosen among those most used:

- ChatGPT (GPT-5) by OpenAI.
- Claude 4.5 by Anthropic.
- GEMINI (2.5 flash) by Google.
- DeepSeek (V3.2-Exp) by DeepSeek.

This approach is displayed in **Figure 1**, while **Figure 2** shows a summary of the programming languages and LLMs we chose.

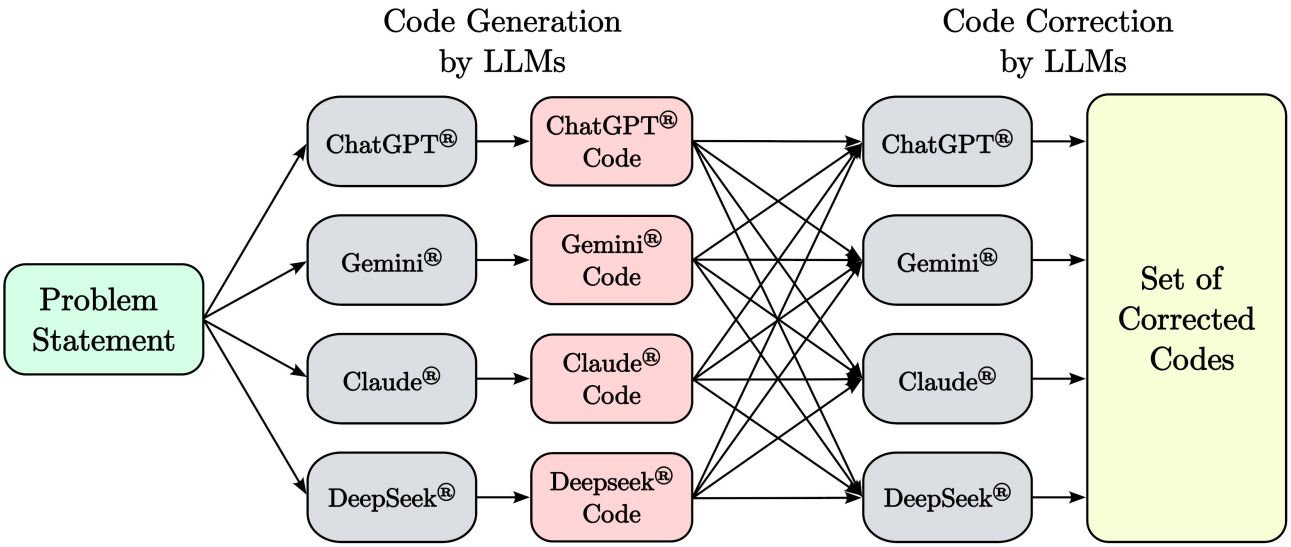


Figure 1 • Graphical description of the methodology followed. Green indicates the problem prompt; gray, the interaction with the large language models; pink, the generated code; and yellow, the corrected version.

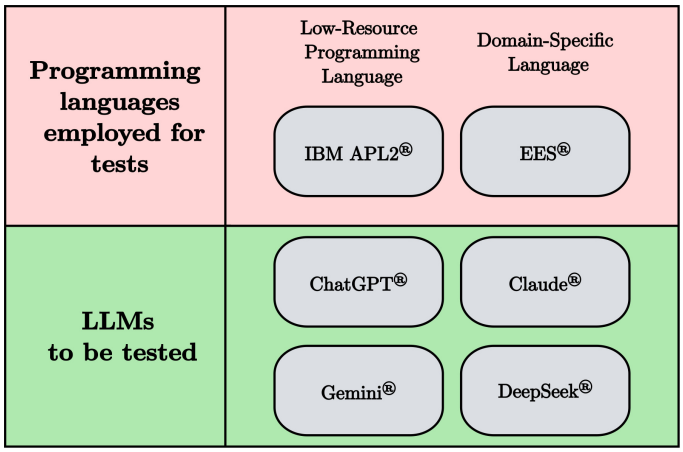


Figure 2 • Summary of programming languages and the large language models chosen to carry out this work. Pink indicates programming languages and green the large language models.

2.1. IBM APL2 as a low-resource programming language

APL (A Programming Language) is a computer language initially proposed in a book by Ken Iverson in 1962 [18], but its structure was not appropriate for computers in those days. In spite of this, in 1964 it was used to describe the architecture of a complete computer (system/360) [19], the first time this was done. A few years later, in 1967, after Iverson's joint work with Adin Falkoff, the language was modified to make it implementable [20], and it was implemented for the IBM/360 machines, making it the first computer language for general use providing functions that worked directly on complete arrays; the first interpreter with efficient dynamic memory allocation and garbage collection routines; and the first time-sharing IBM system working without a human operator, capable of being connected from a terminal via telephone by means of an acoustic modem.

Years later, APL became the first high-level language implemented on a microcomputer (MCM 70, by APL Computers Ltd., in 1973), on an array processor (Analogic, 1983) and on a vector processor (APL2 Release 3, on IBM 3090, in 1987).

The first APL system for the IBM PC (APL/PC) was developed by one of the authors of this paper and a small group working in the IBM Madrid Scientific Center [21]. The group never consisted of more than three people at the same time (occasionally only one), but it endured for sixteen years, during which time several APL interpreters were built. APL/PC was the first product to use the Intel 8087 floating-point co-processor: both APL/PC and the co-processor were announced on the same day, 1 June 1983. In this product, the co-processor was required: the 8087 emulator came later, in 1985, as a part of IBM APL/PC version 2.

The IBM APL2 language [22] was designed and developed during the eighties by James Brown in IBM Yorktown Heights Research Center and Santa Teresa Laboratory and implemented for Intel-based Personal Computers by the IBM Madrid Scientific Center (it was announced in 1988). Other companies and entities, such as Dyalog (Bramley, UK), I.P. Sharp (Toronto, Canada) or GNU (Boston, MA, USA), built their own implementations of APL and APL2 and extended the language in different ways, which gave rise to the apparition of different dialects of the language. This fact makes LLM APL code generation difficult, for the way in which they are built makes them unable to distinguish code written for different dialects.

Since APL and APL2, in whatever implementation, have never been general-use programming languages, we decided that IBM APL2 would be a good representative of the family of LPRL, allowing us to study how LLMs respond to coding questions in the case of this family of languages.

2.2. Engineering Equation Solver as a domain-specific language

EES (Engineering Equation Solver) is a linear and nonlinear algebraic equation solver (although it provides a much wider range of capabilities, such as optimization and parametric study tools) developed by F-Chart Software, a spin-off of the University of Wisconsin–Madison directed by Emeritus Professor Sanford A. Klein. EES consists of an IDE and an associated programming language, which cannot be executed in different IDEs. In fact, some of its capabilities, such as generating plots or carrying out parametric studies, are not accessed by means of code, but by means of the IDE itself, by using a specific Plot menu in the case of plots and by means of Parametric Table Window in the case of parametric studies [23].

The software was originally developed in the context of Thermodynamics and Heat Transfer teaching activities. Solving problems in these fields usually require data about substance properties to be read and interpolated from tables, and large systems of equations need to be solved. EES aims to facilitate the solution of these problems by incorporating a wide range of thermophysical and mathematical function libraries, which have expanded as the software has developed over the years to include more specific libraries for a vast number of mechanical equipment, such as pumps, turbines, combustors, etc. [24].

As previously stated, EES works as an equation solver. In this sense, it uses *equations rather than assignments* [25]. For example, consider the simple code displayed in **Figure 3**: the user writes four lines of code, of which the first two would be considered assignments in other programming languages and the last two would not make sense in most programming languages, since both of them present a variable not previously initialized at the right of the assignment operator. In EES, however, these lines of code are understood as a system of equations, which are solved by means of a Newton–Raphson-based method together with other algorithms which are considered black boxes for the purposes of this explanation [23].

To solve the equations, the solution algorithm employs Newton–Raphson method (a numerical method for solving nonlinear equations and systems of equations that employs a first-order Taylor approximation) [26], combined with sparse matrix tools, which deal with matrices with many zeroes among their elements (in the words of James H. Wilkinson, "*any matrix with enough zeros that it pays to take advantage of them*" [27]), and the Tarjan algorithm, which finds strongly connected components in graphs (defined as the set of vertices that verifies that for every pair of vertices v and w, v can be reached from w and w from v [27]; in EES it is applied to the Jacobian matrix), to decompose the whole system of equations into several subsystems that can be independently solved, improving computation time. EES calls these subsystems "blocks".

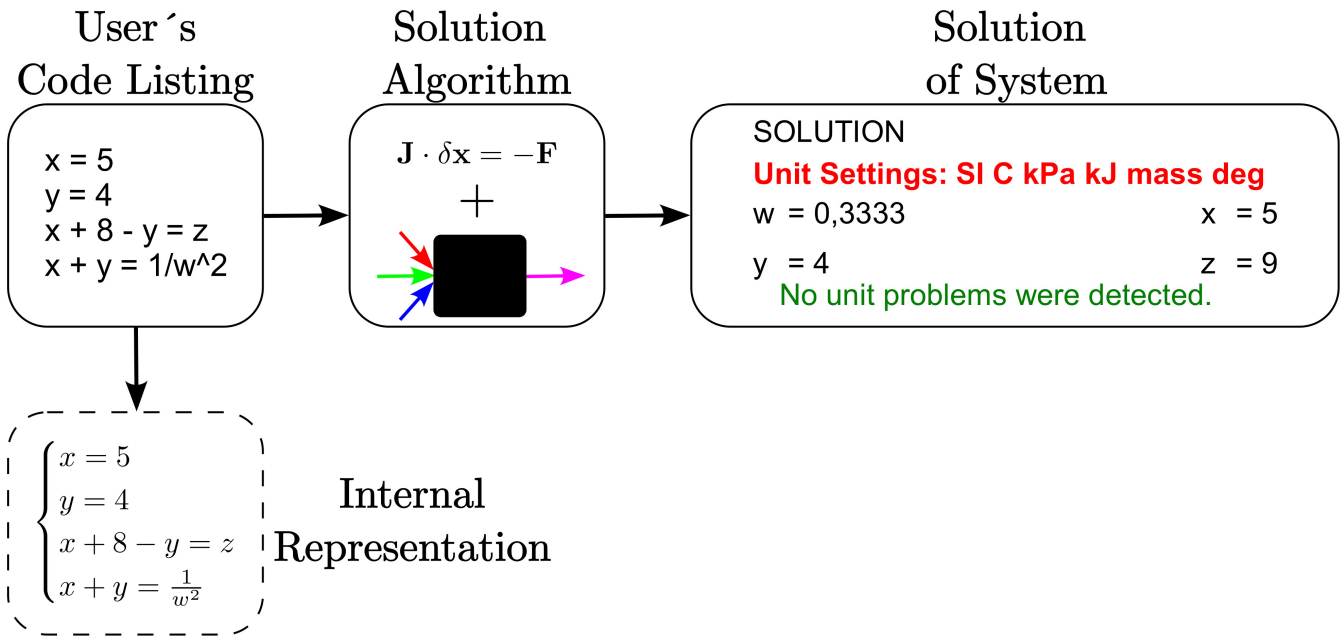


Figure 3 • Rationale behind Engineering Equation Solver. Images under "User's Code Listing" and "Solution of System" titles display screenshots taken from EES IDE in its European version; therefore, commas are used as decimal signs and font colors correspond to those used by EES IDE. Arrow colors used in the image under "Solution Algorithm" title are used to represent a black box algorithm that takes several variable inputs and computes an output.

The solution of the system as presented by the EES IDE is displayed in the right part of **Figure 3** (one interesting feature of EES can be observed in the displayed solution: if not specified by the user, it sets a unit system and carries out a check to verify if there are any unit inconsistencies). It must be noted that EES does not require unknown variables to be solved for: in fact, the fourth equation of the system displayed in **Figure 3** is not solved for w and EES is still capable of finding the solution. Also, equations belonging to the same system of equations do not need to be coded sequentially: EES decomposes the system by aggregating all the equations into decoupled systems when possible.

This way of working is interesting when there are many equations to be solved to perform a simulation and the user wants to experiment with different inputs and outputs, since equations do not need to be rewritten. This allows the user to focus on the modeling part of the work rather than on manipulating the algebraic expressions or in implementing a numerical solution algorithm. For example, consider a Simple Harmonic Motion problem, in which the position x of a particle following that movement is given by

$$x = A \times Sin\,(\omega \times t + \phi_0)$$

where A is the amplitude, ω is the angular velocity, t is time and ϕ_0 is the phase of the oscillator at $t = 0$. In EES, the user can focus on the mathematical description of the problem, without wasting time in algebraically manipulating the equations to solve them for the desired unknown. This is exemplified in **Figure 4**, where the Simple Harmonic Oscillator is solved for different unknowns by selectively commenting one or more lines (note that, since by default the Sin() function expects an argument in degrees and no unit has been declared for the variables, the IDE detects a potential unit problem). In other languages such as Python or MATLAB, the user would have to rearrange the equation to solve for the unknown variable (or use specific equation solving functions).

However, there are some drawbacks to this approach: the user must be careful not to introduce equations which can lead to incompatible systems. An important consequence of this is the fact that *variables cannot be overwritten*. In fact, in EES there is no FOR loop: the closest is "DUPLICATE", which repeats the equations entered as shown in **Figure 5**. Any expression which overwrites a previous one, such as the classical x = x + 1, would lead to an incompatible system of equations (this may not be the case for internal functions or procedures, since DUPLICATE statements are rewritten as REPEAT-UNTIL). When iterative calculations are needed, REPEAT-UNTIL (similar to a DO expression-WHILE condition in other languages) is the right choice.

A lot of emphasis has been placed on the fact that EES is based on equations rather than assignments. This is true for the main program, but EES allows the user to define several types of subroutines, some of which work with assignments rather than equations and, therefore, make it possible to work in a similar way to most programming languages. The following description of these subroutines is mainly based on the EES official manual [23]. **Figure 6** shows an overview of the basic characteristics of different subroutines allowed in EES.

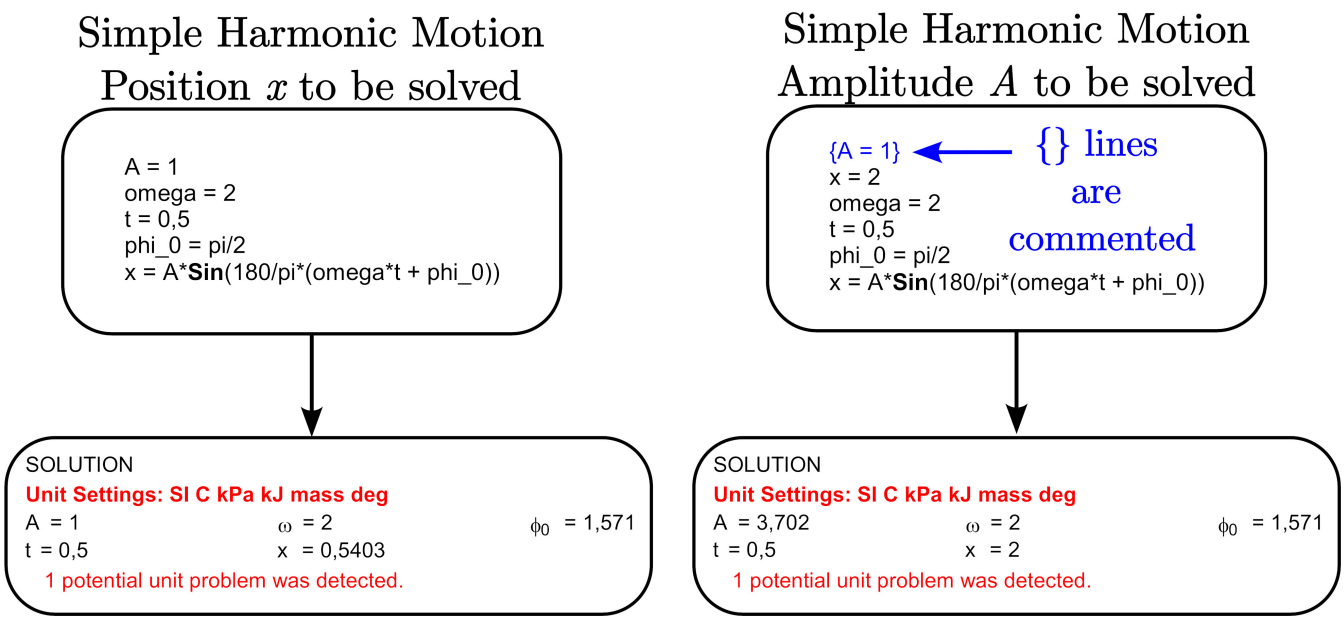


Figure 4 • Example of the benefits of EES way of working. Blue arrows and font are used to highlight commented lines of code. All images displayed in this figure are screenshots of the European version of EES; therefore, comma is the decimal sign.

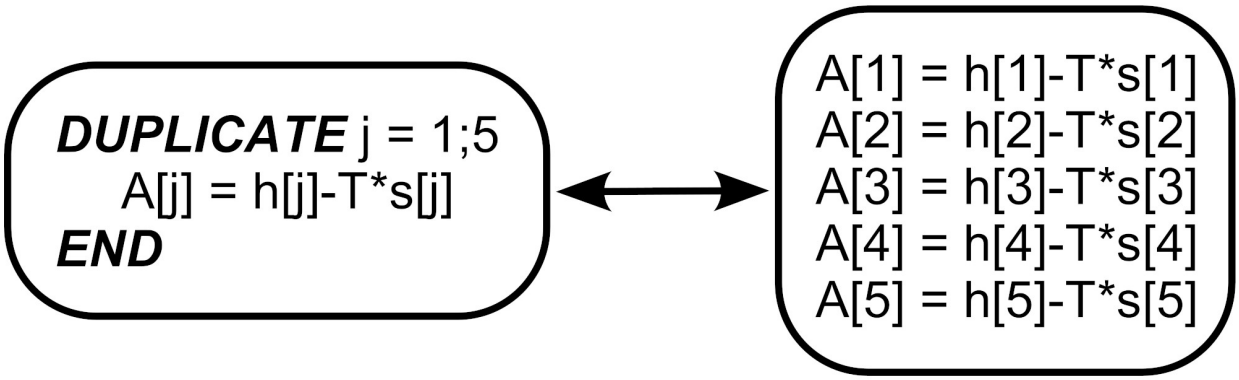


Figure 5 • Example of how DUPLICATE statement works. Both images displayed are screenshots of EES IDE.

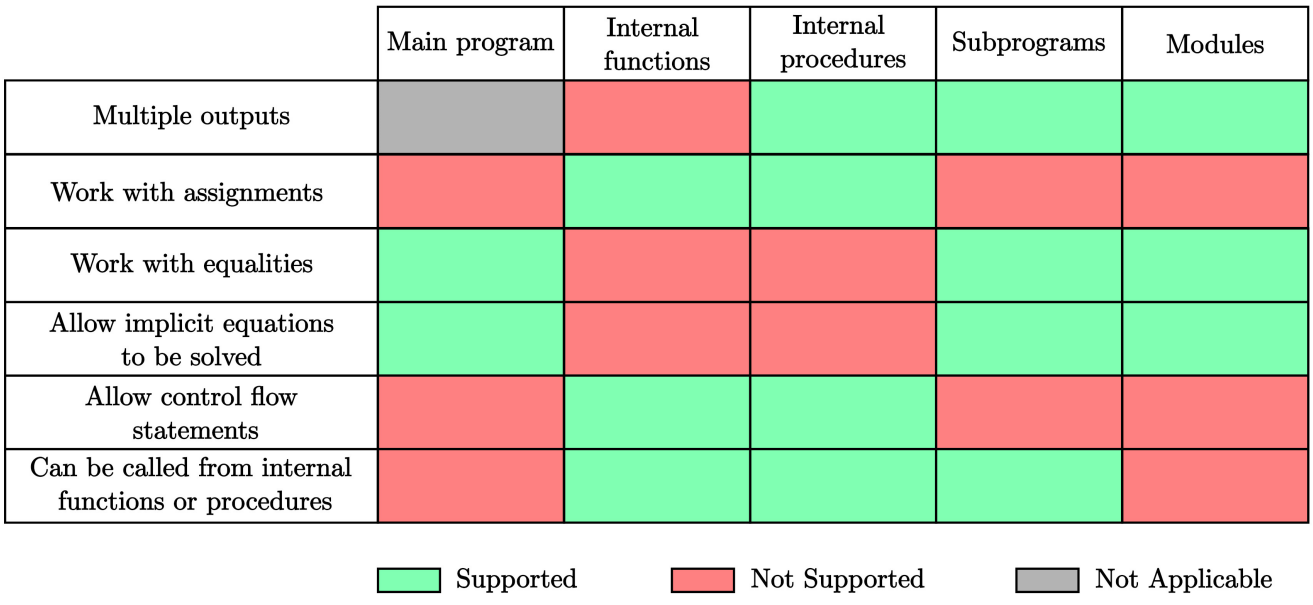


Figure 6 • Overview of basic characteristics of subroutines.

- Internal functions: Subroutines that allow several inputs but just one output, whose statements are assignments rather than equations (the assignment operator in EES is ":=", but the IDE can be configured to accept "=" as assignment operator). Since functions work with assignments, statements such as IF-THEN-ELSE, REPEAT-UNTIL, CASE, GOTO, etc. are allowed. Other functions, procedures and subprograms can be called from a function definition.
- Internal procedures (functions and procedures are identified as "internal" since there also exist "external" functions or procedures, which the user writes in a compiled language such as C, C++, PASCAL, etc. and are called from EES): Similar to internal functions but with several outputs. Inputs are separated from outputs in procedure statements by means of colons (":") (this also applies to subprograms and modules). They also work with statements rather than equations. Functions, procedures and subprograms can be called from an internal procedure.
- Subprograms: These are standalone EES programs (with various inputs and outputs) called from another EES program, which can be the main program, a function, a procedure, another subprogram, etc. These work with equations rather than assignments and, therefore, control flow structures are not allowed. When a subprogram is called from another program, EES creates a new workspace and solves in that workspace the equations of the subprogram.
- Modules: Similar to subprograms, but they cannot be called from functions or procedures due to their internal behavior: modules act like "equation containers" in the sense that equations defined in them are added to the equations of the program from which the module has been called. In [23] it is stated that limited experiments seem to point out that modules are more computationally efficient than subprograms.

This description of EES intends to give the reader an idea of the language to be tested and a prelude to possible mistakes made by LLMs. Since EES is a proprietary software with many capabilities, accessible only by means of its IDE, it operates in a different way than many other languages and it is aimed at specific use cases, so we have decided it is a good representative of the DSL family of languages, allowing us to study how LLMs respond to coding questions in the case of this family of languages.

3. Results

3.1. Code generation in IBM APL2

In the first trial, we posed the same question to the following four LLMs: (a) ChatGPT; (b) GEMINI; (c) DeepSeek; (d) Claude 4.5. The LLM versions used were GPT-5 (ChatGPT), 2.5 flash (GEMINI), V3.2-Exp (DeepSeek) and Sonnet 4.5 (Claude). The question we decided to request, translated into English from the original Spanish we used, was the following:

Write a program in IBM APL2, which works on a word list V and a six-column matrix M of indices to the word list. The matrix reports the number of times a given word appears after a series of four words. The first five columns contain indices to the word list. The sixth column reports the number of times the word in column 5 appears after the four words indicated in the first four columns. Upon receiving four words, the program must search matrix M for their indices in V and add new words from M, chosen from those in the fifth column, with a probability proportional to the number of times they appear (the sixth column of M). Furthermore, once a word is added, the program must continue adding new words using the same algorithm until it finds no more.

Figure 7 shows the solution of the problem, hand-written by one of the authors of this paper. In this solution, the global variable WORDS is the list of words (called V in the request to the LLMs), and the global variable LISTA corresponds to the matrix of indices (called M in the previous description).

```
[0]    CHAT;A;B;I;Z
[1]    L:⎕TC[2],'Make a question with four words'
[2]    →(0=ρA←⍞)/0
[3]    B←⍬WORD1,WORD1,WORD1,WORD1
[4]    L0:I←LISTA[;⍳4]^.=B
[5]    →((I⍳1)>↑ρLISTA)/L
[6]    I←I/⍳↑ρLISTA[;6]
[7]    I←I[((+\LISTA[I;6])≥?+/LISTA[I;6])⍳1]
[8]    →(1=ρ,Z←LISTA[I;5]⊃WORDS)/L
[9]    ⍞←Z,' '
[10]   B←1↓B,LISTA[I;5]
[11]   →L0
```

Figure 7 • Our solution to the IBM APL2 test program for the four large language models. Font colors are those used by IBM APL2 IDE.

This algorithm can be considered as a simplified model of what the LLMs do: in their case, the information is in a neural network, rather than a matrix, but the procedure used to generate strings of words to answer questions by the users is similar. The computing power of the APL2 language can be seen in the fact that the search in a large matrix of a list of four indexes (B) to the last four words in the request, and the answer being generated, can be performed by means of a very simple instruction: instruction [4], which returns a list of zeros and ones of the same length as the number of rows in the matrix LISTA. In the algorithm, instruction [7] randomly selects the next word among those following the current last four words, with a probability related to their abundance. These two instructions [4, 7], make the gist of the algorithm; errors in those two instructions would be critical. The remaining instructions are accessory.

WORD1 in line [3] is an auxiliary function that extracts the next word from the variable A (which is global for WORD1 and local for CHAT), finds it in the variable WORDS (the global list of words), and returns its index. If it is not contained there, the new word is added to the global list. Finally, WORD1 returns the index of the extracted word in the global list.

The first problem we encountered in the four LLMs was their inability to understand that code in IBM APL2 was being requested. They answered with a mixture of code in several APL dialects, using additional structures, unavailable in IBM APL2. GEMINI, for instance, explained it thus, when it was informed that it was falling in that mistake:

> You're pointing out a key difference between the early, purely expression-based style of APL and the Structured Programming features introduced in later implementations, such as IBM APL2 (and others like Dyalog APL).
>
> In traditional, early APL (like APL \360), control flow was primarily handled using the right-pointing arrow for conditional branching: →(condition/expression) followed by the line number or label to branch to.
>
> IBM APL2 (starting in the 1980s) introduced the Structured Programming constructs—often called control structures or structured directives—to make code more readable and maintainable by eliminating the need for explicit line-number branching.
>
> Structured control structures in APL2
>
> The directives used in the previous example (and available in IBM APL2) are:

Directive	Purpose	Traditional APL equivalent
:IF ... :ENDIF	Conditional execution	→(condition/line_label)
:LUP ... :ENDLUP	General loop construct	Loop with conditional → at the end
:LEAVE	Exits the nearest enclosing loop	→line label (to the line after the loop)
:WHILE ... :ENDWHILE	Loop as long as a condition is true	Loop with conditional → at the top
:REPEAT ... :UNTIL	Loop that executes at least once	Loop with conditional → at the bottom

This explanation is wrong. In fact, IBM APL2, the same as the original APL language, does not accept these structures; it uses conditional and unconditional GO TO, represented as →. See a simplified example of this in **Figure 8**.

After we indicated their mistake to the four LLMs, all of them apologized and never again tried to use those structures, whenever they were accessed by the same user from the same IP address. On the other hand, when the other author asked the same question from a different IP address, they made the same mistake again, which means that the problem was not solved in the core LLM, but in the user interface module, so these corrections appeared to depend on the user.

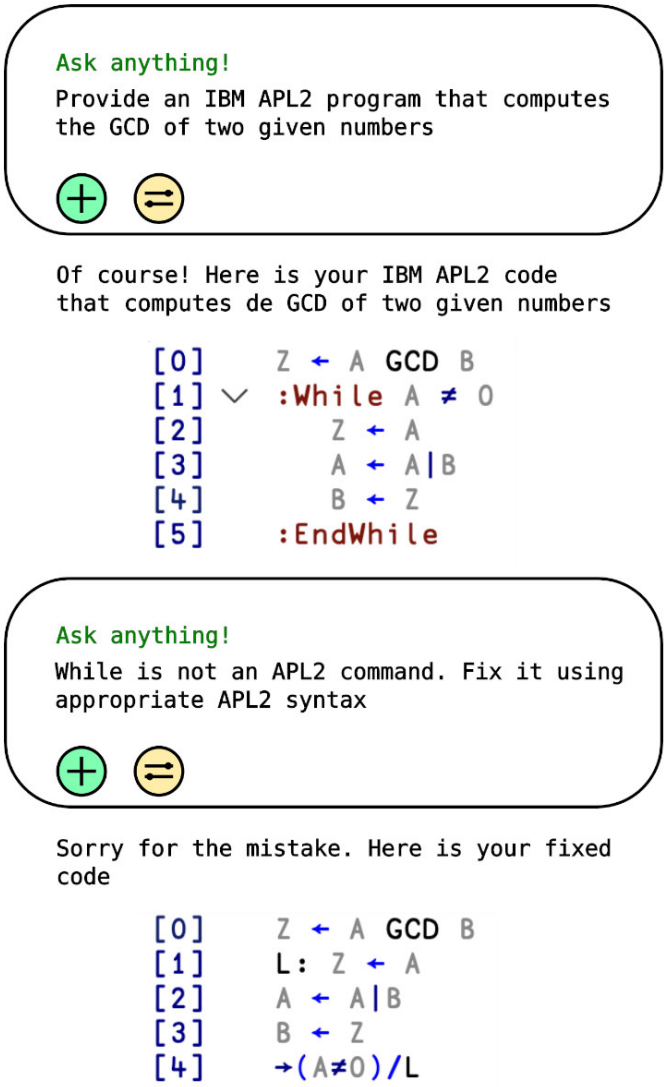


Figure 8 • Screenshot of a simple example of the mistake described in the text, which affected all four large language models tested.

Apart from this, in their second attempt, all four LLMs generated code with several errors, which can be classified in the following groups:

1. Other uses of syntax from different APL dialects.
2. Improper syntax or using a wrong symbol instead of the correct one.
3. A critical error while searching the matrix for the four indices to the words in the prompt, which later, during execution, would have given rise to a RANK ERROR.
4. Different errors while computing the random successor of the last four words. (This is also a critical error.)
5. Miscellaneous errors.

6. The instruction is correct but too complicated (it can be improved).

All four LLMs generated programs with errors numbers 1, 2, 3 and 4, which means that all of them failed to correctly generate the two most important instructions of the algorithm, which correspond to errors 3 and 4 in the previous list, and therefore all of them generated useless programs. An interesting error of the fifth type was generated by DeepSeek: the prompt to ask the user to provide the original four words was repeated twice in the same line, and no attempt was made to make sure that both strings written by the user were identical, which could give rise to an error later.

One interesting problem of type 6, which affected all four LLMs, is the fact that all of them used the quad prompt (⍞) to request the user question, instead of using the quote-quad prompt (⍞). Although this would work, it would make it necessary for the user to enclose the answer to the request in explicit quotes.

The total number of errors in the code generated by the LLMs is indicated in **Table 1** and **Figure 9** (see Supplementary materials for the listings generated and the errors detected manually in them).

3.2. Fixing errors in code written in IBM APL2

In the next set of experiments, we asked the four LLMs fix the errors, if any, in the programs generated by all four LLMs (either by the same one or another). **Table 2** shows the results, where it is apparent that each LLM seemed unable to fix errors in programs generated by itself and scarcely able to correct a few errors in those generated by another LLM.

It can be seen that the error correction was very inefficient. In the best case, ChatGPT corrected two non-critical errors in the GEMINI-generated program, which had five errors, but missed the two critical errors (in fact it detected both, but the corrections offered were invalid). In all the other cases, at most one error was found and corrected, out of three, four, five or eight. In some cases, when the errors were detected but not corrected, alternative options offered were also invalid, sometimes worse than the original instruction being corrected.

In addition, the LLMs pointed out additional supposed errors in correct instructions, and in some cases the suggested corrected instruction was invalid, although in others it was an equivalent or an improved version of the original instruction.

3.3. Code generation in Engineering Equation Solver

Following the methodology described in Section 3.1, we posed a problem to (a) ChatGPT, (b) GEMINI, (c) DeepSeek, and (d) Claude 4.5, with the same problem for all LLMs. As EES is particularly suited for Thermodynamic, Heat Transfer and Fluid Mechanics calculations, the problem was computing the friction factor of a pipe from the Reynolds number, diameter and roughness of the pipe. Translated into English from Spanish, it said this:

Write a code in EES that computes the friction factor of a pipe as a function of Reynolds number, roughness and pipe diameter, differentiating laminar and turbulent cases, both manually and using EES's built-in functions. Output an error message if the results differ and the Reynolds number is not in the flow transition zone considered by EES. If the results differ and it is in that zone, the program should return the value calculated by EES's built-in functions.

Table 1 • Number of errors generated by the large language models.

LLM	Number of errors	Number of critical errors
ChatGPT	8	2
GEMINI	5	2
DeepSeek	4	2
Claude 4.5	3	2

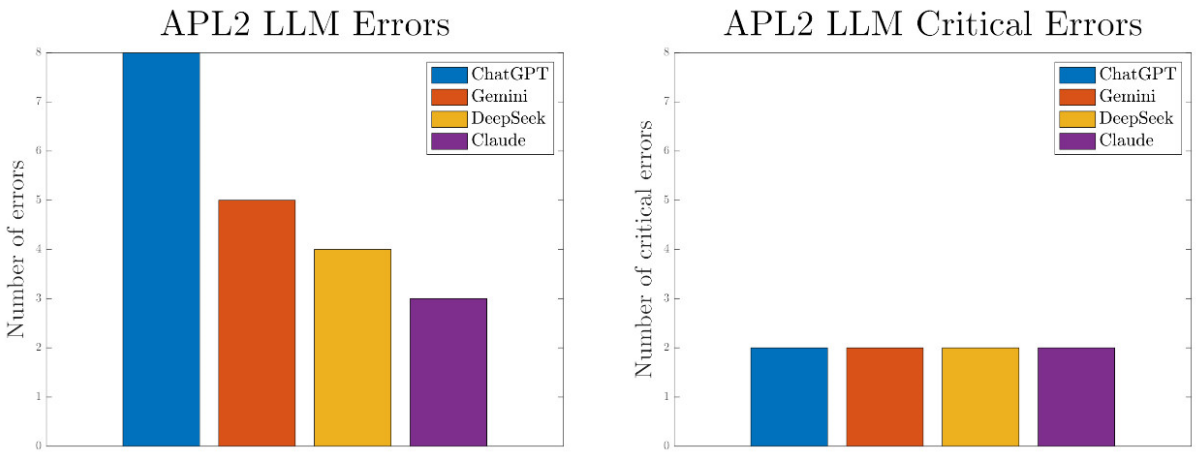


Figure 9 • Summary of the large language models errors for APL2 code.

Table 2 • Number of errors detected and/or corrected by the large language models.

LLM/LLM	Nr. of errors detected & corrected	Critical errors corrected
ChatGPT by ChatGPT	0	0
ChatGPT by GEMINI	1	0
ChatGPT by DeepSeek	1	0
ChatGPT by Claude 4.5	1	0
GEMINI by ChatGPT	2	0
GEMINI by GEMINI	0	0
GEMINI by DeepSeek	1	0
GEMINI by Claude 4.5	1	0
DeepSeek by ChatGPT	0	0
DeepSeek by GEMINI	1	0
DeepSeek by DeepSeek	0	0
DeepSeek by Claude 4.5	0	0
Claude 4.5 by ChatGPT	1	0
Claude 4.5 by GEMINI	0	0
Claude 4.5 by DeepSeek	0	0
Claude 4.5 by Claude 4.5	0	0

The schematic representation of the problem to be solved is shown in **Figure 10**. This involves checking if the Reynolds number provided categorizes the flow as laminar (Re < 2300) or turbulent (Re > 3000) according to EES criteria. If laminar, the friction factor is computed as $\frac{64}{Re}$, which corresponds to the friction factor for laminar flow in a circular pipe, while in the turbulent case, computing the friction factor involves solving Colebrook–White equation [28], defined as $\frac{1}{\sqrt{f}} = -2 \cdot log\left(\frac{\frac{\epsilon}{D}}{3.7} + \frac{2.51}{Re \cdot \sqrt{f}}\right)$. If 2300 < Re < 3000, the flow is said to be in a transition regime and, for the purposes of this paper, the friction factor is computed following EES criteria of interpolating between laminar and turbulent values.

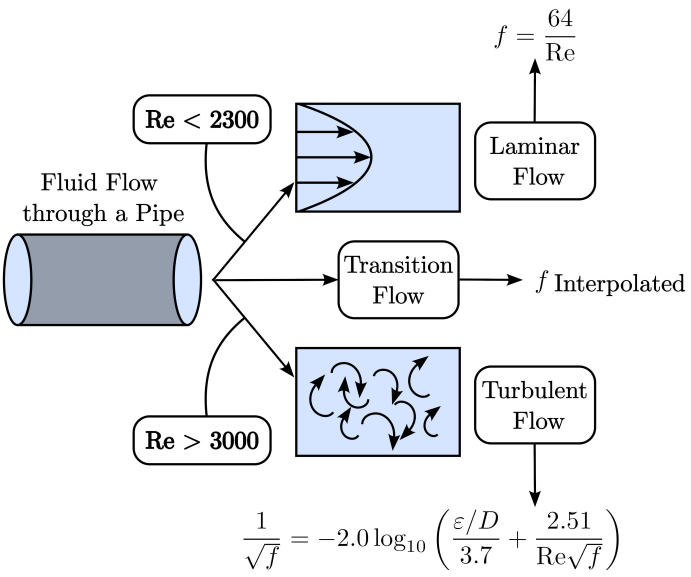


Figure 10 • Schematic representation of the problem to be solved.

Checking the flow regime involves an IF-THEN-ELSE statement, which cannot be placed on the main program, and the Colebrook–White equation must be solved iteratively, which can be done explicitly by means of a fixed-point iteration (this requires a DUPLICATE-UNTIL statement) or with EES equation solving capabilities, in which case the equation must be placed outside the function or procedure where the flow regime is checked. In addition, EES provides a pre-defined function to compute the friction factor, PipeFlow_ND (functions and procedures are identified as "internal" since there also exist "external" functions or procedures, which the user writes in a compiled language such as C, C++, PASCAL, etc., and are called from EES). In [29], detailed explanations of correlations employed by PipeFlow_ND and solved examples using EES are presented. So the problem posed to the LLMs aims to check whether these systems can identify the casuistry regarding EES subroutines capabilities and predefined functions. The problem itself poses no difficulty when the equations are known, as is the case here, but criteria for determining when the flow is turbulent are somewhat diffuse and LLMs should generate code coherent with the criteria employed by EES.

Figure 11 shows the solution of the problem, hand-written by one of the authors of this paper, which uses a procedure to encapsulate the code. To illustrate the different alternatives allowed by EES, an equivalent program is presented in Supplementary materials, Figure S1, which uses a combination of a function and a subprogram.

```
{PROCEDURE DEFINITION}
PROCEDURE friction(epsilon; D; Re; tolerance; MaxIter:f; NumIter; RelError)
  NumIter = 0
  {Hand computation}
  IF (Re <= 2300) THEN
     {Laminar flow case: f = 64/Re}
     f = 64/Re
     RelError = 0
  ELSE
     {Turbulent flow case: f obtained by iterating Colebrook-White}
     {Brute-force iteration}
     f = 0,02
      REPEAT
        f_nuevo = 0,25 / (Log10((epsilon/D)/3,7 + 2,51/(Re*Sqrt(f))))^2
        RelError = Abs((f_nuevo - f)/f)
        f = f_nuevo
        NumIter = NumIter + 1
    UNTIL ((RelError < tolerance)  or (NumIter >= MaxIter))
  ENDIF
 {Computation using libraries}
 {Data preparation}
 RelRough = epsilon/D
 LoverD = 1e6
 Pr = 1
 CALL PipeFlow_ND(Re;Pr;LoverD;RelRough: Nusselt_T;Nusselt_H;f_check)
 {Check}
 IF (Round(f - f_check; 0,0001) <> 0) THEN
    IF ((Re > 2300) and (Re < 3000)) THEN
       f = f_check
    ELSE
       CALL Error('Hand-computed friction factor differs from library computed one. Improve tolerance')
    ENDIF
 ENDIF
END
```

Figure 11 • Our solution to the Engineering Equation Solver test program suggested to the four large language models.

This algorithm uses a procedure which accepts as inputs the pipe roughness (ϵ), the diameter (D) and the Reynolds number (Re) as requested in the problem statement, together with a tolerance for iterations (tolerance) and the maximum number of iterations to carry out (MaxIter). These iterations correspond to those required to solve the Colebrook-White equation. The program returns as outputs the friction factor (f), the number of iterations which have been required (NumIter) and the relative error achieved by iteration i = NumIter versus the preceding iteration. Regime checking and comparison between hand-computed and EES-computed friction factors is carried out inside various IF-THEN-ELSE statements. Colebrook-White's iteration is carried out in a REPEAT-UNTIL loop, since procedures work with assignments rather than equations and, therefore, no implicit equations can be directly solved. For the purposes of simplifying the code, we decided to compute the transition regime as if it were turbulent and let the program use the EES-computed value when in transition regime.

The most relevant problem found after posing the problem to the LLMs is the fact that they do not employ subroutines to develop the requested code, immediately yielding programs which cannot compile, due to errors. One exception to this tendency is the second code generated by DeepSeek: as will be explained in detail in the following paragraphs, when asked to solve the problem, DeepSeek offered two alternative codes (which were not really alternative, since the first one did not use a subroutine), the second one employing a procedure. However, DeepSeek was not able to identify that these two programs were not interchangeable. Another surprising problem was the tendency of LLMs to use invented functions to solve the problem. The most relevant was "FrictionFactor", an invented function employed by all LLMs but GEMINI (which instead called the function "Colebrook", also invented) to compute the friction factor on the ground that this is the predefined EES function to do so. This is surprising, as there exists an online official public guide advising how to use PipeFlow_ND and this data should be accessible by LLMs.

The errors can be classified into the following groups:

1. Including IF-THEN-ELSE statements in the main program.
2. Attempting to directly solve (as they would be solved in the main program) implicit equations inside IF-THEN-ELSE statements.
3. Using non-existent built-in functions.
4. Using non-existent flow-control structures, such as ELSEIF, BREAK or FOR.
5. The transition regime being incoherent with EES criteria.
6. Attempting to display results using code.
7. Miscellaneous errors.

Errors 1 and 2 can be seen as two sides of the same coin: since LLMs generate code without considering whether a subroutine or the main program is adequate, and encapsulate everything in the main program, generated code mixes characteristics from both "equation subroutines" (main program, subprograms and modules) and "assignment subroutines" (functions and procedures). Error 1 was common to all four LLMs. DeepSeek, when asked to solve the problem, offered two alternative codes, the first one encapsulating the code in the main program and the second one in a procedure. Only the first of these codes contains Error 1, but it is somehow latent in the second one, as DeepSeek said the two codes were equivalent. Error 1 is common to ChatGPT and Claude.

Errors 3 and 4 could be categorized as typical hallucinations, since LLMs use functions or programming structures that do not exist in EES. As explained above, the most paradigmatic example of an invented function is "FrictionFactor", while other examples are "Write" and "InfoError", which will be explained in detail in the following paragraphs. Invented programming structures categorized as Error 4 include ELSEIF (the EES user guide [23] clearly states: "The Else keyword should appear on a line by itself, followed by the statements which execute if the conditional test is false," which means that ELSEIF can be constructed by using ELSE + Line break + IF), an error common to ChatGPT, GEMINI and DeepSeek I. Other structures wrongly generated by LLMs include FOR and BREAK. Error 3 is common to all four LLMs, and Error 4 to all but Claude.

Error 5, as advanced in the problem statement, is related to the transition range EES considers, which may differ from the criteria some textbooks employ [28]. However, the authors consider that generating code coherent with EES criteria falls under generating correct code. We have observed that all generated codes use an incorrect criterion, as they consider that turbulent regime begins at Re = 4000, which is a common criterion, but not the one needed in this case.

Error 6 can be considered as a subset of 3, since the functions LLMs employ to display results do not exist in EES, but the authors think it is better to differentiate both kinds of errors. In EES, results are automatically displayed after program execution in the Solution Window and the Arrays Window [23], so trying to display them is a fundamental mistake (there exists a PRINT command in EES but it can only be used in internal functions and procedures and it is used to write results in an external file, such as a .txt or .csv file).

In general, errors in the logic of the programs were not relevant, but the generated code seems to take elements from several programming languages, which can be observed for example in the use of FOR, BREAK and ELSEIF statements, attempting to display results using code, etc. Additionally, it is clear from these set of experiments that LLMs still require the user to provide information about the conventions employed by functions, as has been shown for the laminar–turbulent flow transition range.

The total number of errors in the code generated by the LLMs is indicated in **Table 3** and **Figure 12** (see Supplementary materials for the listings generated and the errors detected manually in them). It must be noted that, since EES requires compilation to run, syntax errors are likely to be critical. Therefore, these numbers must not be directly compared with those presented in **Table 1**.

Table 3 • Number of errors generated by the LLMs.

LLM	Number of errors	Number of critical errors
ChatGPT	8	6
GEMINI	6	3
DeepSeek 1	11	8
DeepSeek 2	7	4
Claude 4.5	6	4

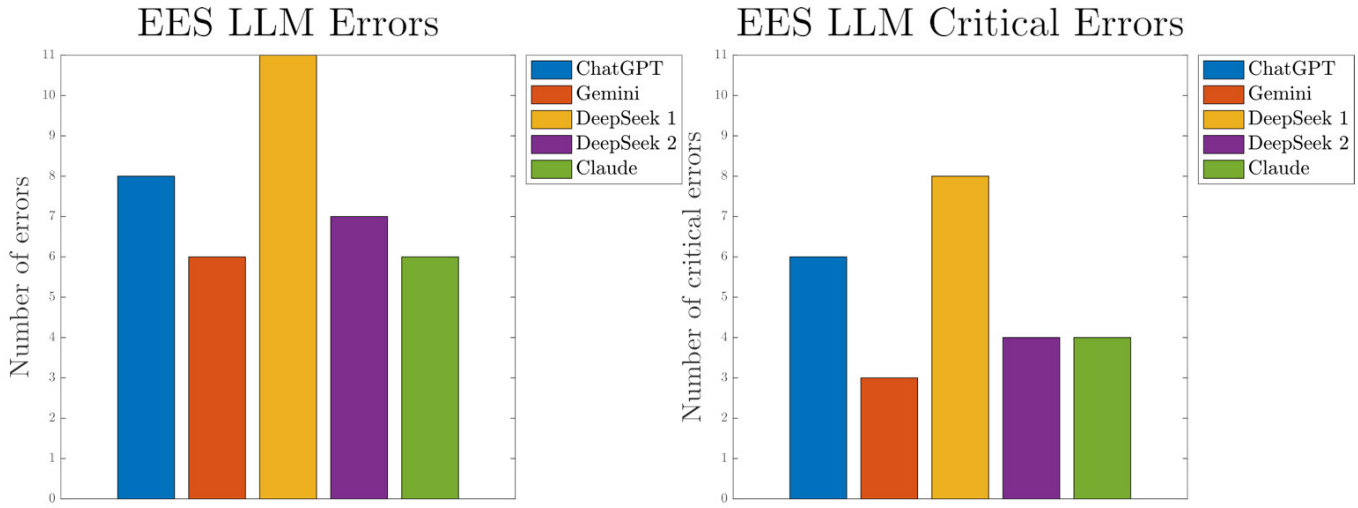


Figure 12 • Summary of the large language models' errors for Engineering Equation Solver code.

3.4. Fixing errors in code written in Engineering Equation Solver

Following the methodology described in Section 3.2, we asked the four LLMs to fix the errors, if any, in the programs generated by all four LLMs (either the same or another), obtaining a set of twenty corrections, since DeepSeek provided two codes and we tested four LLMs. **Table 4** shows the results, which do not differ substantially from those presented in Section 3.2 for the analogous experiment with APL2. For the purposes of this table, Error 5 has been ignored, as the LLMs were provided with the code, but not with the initial problem statement, so they could not know we were asking for EES criteria. When an error categorized as 7 (miscellaneous error) has to do with the problem statement, the same criterion is applied.

As advanced in the precedent paragraph, the results are very similar to those related to APL2, with the LLMs providing very inefficient corrections. In fact, all LLMs identified as errors parts of the code which were not incorrect, thus introducing additional errors to the code, or correctly identified as erroneous some lines of code but provided an incorrect alternative. Interestingly, an alleged error which appeared in almost all corrections was related with the use of upper- and lower-case letters: while it is true that EES code usually presents flow-control statements and other functions written with upper-case letters, writing them in lower case is not a problem, since the IDE corrects this when the Check button is pressed. This is done previously to compiling to check syntax, and to check that the number of equations equals the number of unknowns. In this case, an incorrect correction of LLMs does not introduce additional errors to the code.

Another interesting alleged error is related to comments, which in EES can be written with curly brackets ({}) or quotation marks (“”). LLMs tended to point out that one or both ways to write comments were incorrect, and proposed alternatives, introducing additional errors.

4. Discussion

A set of experiments evaluating the correctness of code generated by LLMs in two languages (one LRPL and one DSL) was carried out. The LLMs chosen for this study were ChatGPT by OpenAI, Claude 4.5 by Anthropic, GEMINI by Google and DeepSeek by DeepSeek. The LLM versions used were GPT-5 (ChatGPT), 2.5 flash (GEMINI), V3.2-Exp (DeepSeek) and Sonnet 4.5 (Claude). In the four cases we used the LLMs without customizations. Our tests were carried out in October 2025.

Table 4 • Number of errors detected and/or corrected by the LLMs.

LLM/LLM	Nr. of errors detected & corrected	Critical errors corrected
ChatGPT by ChatGPT	3	2
ChatGPT by GEMINI	2	1
ChatGPT by DeepSeek	1	0
ChatGPT by Claude 4.5	1	0
GEMINI by ChatGPT	0	0
GEMINI by GEMINI	0	0
GEMINI by DeepSeek	1	1
GEMINI by Claude 4.5	1	1
DeepSeek 1 by ChatGPT	2	1
DeepSeek 1 by GEMINI	0	0
DeepSeek 1 by DeepSeek	0	0
DeepSeek 1 by Claude 4.5	0	0
DeepSeek 2 by ChatGPT	0	0
DeepSeek 2 by GEMINI	0	0
DeepSeek 2 by DeepSeek	0	0
DeepSeek 2 by Claude 4.5	2	2
Claude 4.5 by ChatGPT	1	1
Claude 4.5 by GEMINI	0	0
Claude 4.5 by DeepSeek	0	0
Claude 4.5 by Claude 4.5	1	1

IBM APL2 was chosen as a representative of LRPL and EES as a representative of DSL. IBM APL2 is a good representative of the family of LRPLs:

- It is undoubtedly a little-used language.
- It is a very concise language, which evolved from a mathematical notation, with many powerful properties easily out of the reach of LLMs.
- There are many dialects similar to this language, compatible to a certain point, but incompatible in other scenarios.

EES is a good representative of the family of DSLs:

- Even though it may be used as a general-purpose equation solver, it is mainly intended to be used to solve problems in Thermodynamics, Fluid Mechanics and Heat Transfer, as is indicated by the wide range of the libraries designed for this usage.
- Its programming paradigm (equation solving in the main program and relying on subroutines for flow-control structures and more traditional programming tasks) is different from general-use languages and is designed for specific usages.
- The language is intrinsically dependent on the IDE and the commercial software product is a combination of both.

For each language, all four LLMs were provided with a common problem statement and asked to generate a code for that problem, thus obtaining a set of LLM-generated codes. Subsequently, every code was given to all LLMs, pointing out that it had errors, without specifying them, and asking LLMs to fix the codes. The analysis carried out intends to provide a first taxonomy of errors found in LLM-generated code for LRPL and DSL, without aiming to develop statistical benchmarks of functional correctness.

The results of the experiments described in Section 3 indicate that the LLMs did not generate code satisfactorily in the cases of the little-used languages and domain-specific languages we used. In fact, most LLM code generation studies deal with highly used languages such as Python, where code generation can be more useful, because the Internet contains a much larger amount of information in their regard. Only one of the references we have reviewed [14] addresses the two families of languages we have considered, although they focus primarily on benchmarks and metrics used to measure the efficiency of automated programming in those languages.

A recent paper [30] in the same direction as ours, addressing the generation of Python code, emphasizes the fact that metrics for evaluating LLM code should not be limited to formal correctness, and presents a prompting and testing strategy similar to ours. Our setup is similar, as it focuses on evaluating in detail a small number of responses generated by LLMs, using a zero-shot prompting strategy.

In our tests, one error we detected repeatedly has to do with incorrect syntax of the language addressed, which manifests itself as mixing different versions of the language in the IBM APL2 case and using invented built-in functions and statements in the case of EES. We believe this situation may extrapolate to other languages, even those more frequently used, as will be described in Section 4.1.

Another interesting feature of code generated by LLMs is their inability to identify actions that can only be addressed through an IDE. As explained in Section 2, EES relies on its IDE to generate plots or automatically display results, while the LLMs tried to perform this by means of code.

Regarding the second part of the experiments carried out, which dealt with iterating generated code through the same LLM that generated it or a different one, the results obtained indicate that these corrections did not detect most errors and, in fact, introduced new errors in most cases. It has been noticed that LLMs adapt the way in which they generate code when feedback is suggested by the user, but this improvement seems to be available only for users connected to the same IP address as that of the user who provided feedback.

All this is especially important when LLM code generation is applied to education in the field of computer programming, as shown in reference [17], where it has been observed that students tend to accept automatically the programs offered by the LLMs, and sometimes do not even try to compile them, or execute them when they compile correctly, to see if they perform as desired. Even if the programs execute correctly, functional correctness is not the only dimension to evaluate whether a program is correct or not.

In this context, the author of a Python manual asserted that the absence of errors is less important than the fact that the code is written in such a way that the users (or students) may find the errors by themselves. The actual quotation is found in reference [31], and the specific sentence is: “When a program appears to be doing the right thing but isn’t, bad things can follow: fortunes can be lost, patients can receive fatal doses of radiation therapy, airplanes can crash. Whenever possible, programs should be written so that when they don’t work properly, it is self-evident”.

LLMs usually generate code well-provided with comments, but sometimes this is not enough. And the fact that people tend to trust implicitly anything generated by what is called “artificial intelligence” may become a big stumbling block. After carrying out the set of experiments described in Section 3, we believe the use of reference manuals and user guides is still a key part of software development, at least in LRPL and DSL.

When these systems are applied in programming education, they could be improved by incorporating an option to enable or disable the acquisition of information by the LLMs from their conversation with the user. The instructor should be allowed to provide feedback to the LLM, but the students should not provide “noisy” feedback, not related to best practices. An example of good practice would be to include a docstring in every user-defined function to help code legibility, inform about the units of the inputs of the function, etc. This would not be difficult. LLMs consist of a neural network, where a lot of information is stored but there is little “intelligence”, and an entry algorithm, similar to that used in Section 3.1 as a problem to solve by the LLMs. This algorithm includes a module that modifies the questions to the neural network based on previous experience with each user (without passing information from one user to another, as we have

seen). This module is a classic program and therefore can be easily modified in the sense suggested.

Despite their flaws, LLMs are probably being overused, but the previous discussion may be a compromising solution, to ensure that they meet certain requirements. It has been said that the answers provided by LLMs are useful only to those who already know the answer and actually did not need to ask the question. This may be an exaggeration, but it encloses a pinch of truth.

Current use could lead us to a vicious circle where LLMs learn the languages most used by people, and people use the languages where LLMs generate the best code. It would be a shame if interesting languages were left out because of this. **Figure 13** summarizes this vicious cycle.

4.1. General failure modes of code-generating large language models

Syntax errors and those errors that make the generated program fail to obtain the expected value can happen when code is generated for every language. In general-use languages, it is not too difficult to find out whether the syntax of the generated code corresponds with the definition of the language, or whether the result of the execution is the same as predicted. On the other hand, the two families of languages we have considered here give rise to additional considerations. Here we will consider the two cases we have discussed and try to deduce an error taxonomy beyond the particular languages we have selected and discussed here.

The problem with little-used languages (LPRLs) is that this category is not well-defined. There are many languages that should be classified as little-used, and they span the whole range of computer languages: low-level and high-level languages; declarative versus imperative languages; logic languages; functional languages (such as LISP); object-oriented languages (such as Smalltalk); scripting languages like R; compiled or interpreted languages; and general-purpose or specific-purpose languages.

In little-used languages that are available in multiple dialects, if LLMs are used to generate code, one error that can be expected is the mixed use of different dialects in the same code. This error (dialect conflation) would make the code fail on compilation in most cases. This situation can also be applied in the following cases:

- Old languages that have changed their standards over time, such as FORTRAN. A typical response of an LLM would be to mix different standards in the generated program.

- Clonic languages, such as Scilab, which is highly based on MATLAB. As MATLAB is more widely used and there is much more available information, when asking an LLM to generate Scilab code, it may use syntax valid only in MATLAB. This problem can also appear in languages which are highly dependent on external libraries such as Python, if they are provided with libraries that are changed or actualized over time, or are not as popular as those more highly used.

Domain-specific languages (DSLs) are a different category. In many cases, they interact with the programmer through an Integrated Development Environment (IDE), which, being outside the verbal capabilities of LLMs, is very difficult to be considered by these computing tools.

- LLMs are usually unable to identify actions that can only be addressed through an IDE (IDE–language boundary confusion). This situation may get worse in languages in which the IDE becomes relevant in program development, as is the case for example of programs such as Simulink, a graphical programming language based on MATLAB, or OOCSMP, an object-oriented simulation language designed by one of the authors of this paper [32].

- There is a further difficulty if the language uses especial programming paradigms, as in the case of EES, where the objective of the main program is to solve equations, rather than execute control structures, as in most other languages.

- Another important problem would be the use by LLMs of invented built-in primitive functions and statements. DSLs may include functions for very specific problems, such as PipeFlow_ND in the case of EES (whose usage is exemplified in **Figure 11**), which are not common to general-use languages and which are expected to constitute a source of errors in LLM-generated code.

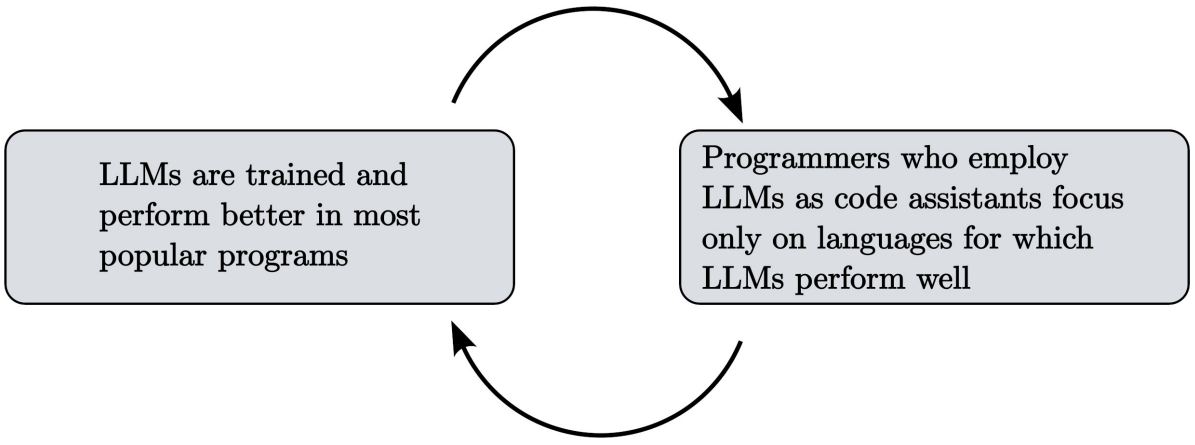


Figure 13 • Vicious cycle that could be created with large language models and programming languages.

In embedded systems, where the hardware is more important than the language version, as each microprocessor has its own characteristics, LLMs may be unable to take those features into account.

Besides this, it must be noted that LRPL and DSL are not relics of the past: consider for example Programmable Logic Controllers (PLCs). These computers are programmed employing highly standardized DSLs, such as Structured Text or Ladder Logic (which happens to be a graphic programming language), and a vast proportion of the tools we use daily depend on PLCs: elevators, electric measurement devices, HVAC (Heat, Ventilation and Air Conditioning) systems, etc. This example can be used to see that even those languages which seem to be old-fashioned or specific play a role in our society.

4.2. Limitations

This paper reports the application of four different LLMs to generate code in two specific testbeds, respectively belonging to the two language families (LPRL and DSL) considered in our investigation. We have detected significant failures both in code generation and code correction by the four LLMs. To a certain extent, this result was to be expected, but we consider that the experiment had to be conducted to confirm the predictions, because evidence in this area is sparse. Furthermore, our analysis provides valuable insights for EES and APL2 programmers by highlighting common pitfalls in LLM-generated code. This allows them to watch out for these errors in their LLM-generated code or to address them directly through more precise prompting which incorporates the information presented in this paper.

This work has the following limitations:

(a) A small sample size: We have analyzed just two languages (one per family) and just two problems (one per language). Given more time, a larger number of test problems could be offered to the four LLMs, and the same procedure could be applied to more languages in both families. We consider this as future work.

(b) If we had a sufficient number of cases, formal statistical procedures could be applied to evaluate significance tests for the four LLMs, to determine if the observed effects are genuine or due to chance.

However, it must be noted that there exists a trade-off between statistical significance of tests and the depth of the code correction task performed by the authors: generating a large number of LLM codes would require to restrict the evaluation of code to its functionality or, at least, reduce the level of detail with which code has been assessed in this paper. We consider both approaches, statistical tests and detailed corrections, relevant but in this paper we have chosen to follow the second.

5. Conclusions

The experiments described in Section 3 of this article demonstrate that LLMs do not currently generate useful code, when requested to use low-resource and domain-specific programming languages. As indicated in Section 4, the fact that the most frequently used languages are associated with a huge amount of information on the Internet, which can be used, and in fact has been used, to train the LLMs, can have an undesirable effect on the use of different languages by programmers, by reducing the programming language diversity and pushing most programmers to use only the most frequently used languages, where LLMs are expected to generate better code.

A few suggestions are offered to improve the educational use of LLMs to teach computer programmers, where bad use of these tools can be harmful.

In the future, this study could be improved in three ways:

- By using the four LLMs we have used with other problems in the same languages we have used here (IBM APL2 and EES) so that we would be able to generate statistics which complement the error taxonomy developed in this study. It must be noted, however, that the problems in conducting that statistical analysis should ideally exploit the peculiarities of IBM APL2 and EES in order to provide insightful conclusions, so a first step should be to figure out what is a good test problem for these languages.

- By applying the same procedure to different low-resource languages and domain-specific programming languages. In the case of DSLs, this category of languages is intended for use by experts, or at least users with in-depth knowledge, and it is unrealistic for a single person to master a large number of them. Therefore, conducting an in-depth study would require a multidisciplinary team of experts who would subject LLMs to tests specific to their fields of knowledge and to the respective languages.

- By applying the same procedure to higher-use languages different from Python, C++ or Java, to find out possible differences between code generation styles in special cases. MATLAB, which may be categorized as the boundary between LRPL and higher-resource programming languages, which has a long development history, offers a vast public documentation and is still the preferred choice in several industries due to built-in toolboxes, and it may offer a good comparison point to determine if challenges identified for these kind of languages correlate with user base.

Acknowledgments

The authors thank Professor José Ignacio Linares (Universidad Pontificia Comillas) for his valuable insights on EES and his views on what would be a good problem statement to evaluate LLMs.

Funding

This research received no external funding.

Author contributions

Conceptualization, M.A. and A.M.C.; methodology, M.A. and A.M.C.; software, M.A. and A.M.C.; validation, M.A. and A.M.C.; formal analysis, M.A. and A.M.C.; investigation, M.A. and A.M.C.; data curation, M.A. and A.M.C.; writing—original draft preparation, M.A. and A.M.C.; writing—review and editing, M.A. and A.M.C.; visualization, M.A. and A.M.C. All authors have read and agreed to the published version of the manuscript.

Conflict of interest

The authors declare that they have no competing interests.

Data availability statement

All data supporting the findings of this publication are available within this article and/or its supplementary materials.

Supplementary materials

The supplementary materials are available at https://doi.org/10.20935/AcadAI8228.

Additional information

Received: 13 November 2025

Accepted: 21 March 2026

Published: 31 March 2026

Academia AI and Applications papers should be cited as *Academia AI and Applications 2026*, ISSN 3071-0286, https://doi.org/10.20935/AcadAI8228. The journal's official abbreviation is *Acad. AI*.

Publisher's note

Academia.edu Journals stays neutral with regard to jurisdictional claims in published maps and institutional affiliations. All claims expressed in this article are solely those of the authors and do not necessarily represent those of their affiliated organizations, or those of the publisher, the editors and the reviewers. Any product that may be evaluated in this article, or claim that may be made by its manufacturer, is not guaranteed or endorsed by the publisher.

Copyright

© 2026 copyright by the authors. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

References

1. Cox B. Object-oriented programming: an evolutionary approach. Reading (MA): Addison-Wesley; 1986.
2. Koza JR. Genetic programming: on the programming of computers by means of natural selection. Cambridge (MA): MIT Press; 1992.
3. O'Neill M, Ryan C. Grammatical evolution, evolutionary automatic programming in an arbitrary language. Dordrecht, New York (NY), Norwell (MA), London: Kluwer Academic Publishers; 2003.
4. Alfonseca M. Automatic generation of APL programs. APL Quote Quad. 2007;35(4):21–5. doi: 10.1145/1354923.1354926
5. Igwe K, Pillay N. Automatic programming using genetic programming. Proceedings of the 2013 Third World Congress on Information and Communication Technologies (WICT 2013); 15–18 December 2013; Hanoi, Vietnam. p. 337–42.
6. Zhao WX, Zhou K, Li J, Tang T, Wang X, Hou Y, et al. A survey of large language models. arXiv. 2023. arXiv:2303.18223. doi: 10.48550/arXiv.2303.18223
7. Naveed H, Khan AU, Qiu S, Saqib M, Anwar S, Usman M, et al. A comprehensive overview of large language models. ACM Trans Intell Syst Technol. 2025;16(5):1–72. doi: 10.1145/3744746
8. Nam D, Macvean A, Hellendoorn V, Vasilescu B, Myers B. Using an LLM to help with code understanding. Proceedings of the IEEE/ACM 46th International Conference on Software Engineering; 14–20 April 2024; Lisbon, Portugal. p. 1–13.
9. Wang J, Chen Y. A review on code generation with LLMs: application and evaluation. Proceedings of the 2023 IEEE International Conference on Medical Artificial Intelligence (MedAI); 18–19 November 2023; Beijing, China. p. 284–9.
10. Zamfirescu-Pereira JD, Jun E, Terry M, Yang Q, Hartmann B. Beyond code generation: LLM-supported exploration of the program design space. Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems; 26 April–1 May 2025; Yokohama, Japan. p. 1–17.
11. Zhao S, Zhu A, Mozannar H, Sontag D, Talwalkar A, Chen V. CodingGenie: a proactive LLM-powered programming assistant. Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering; 23–28 June 2025; Trondheim, Norway. p. 1168–72.
12. Dong Y, Jiang X, Qian J, Wang T, Zhang K, Jin Z, et al. A survey on code generation with LLM-based agents. arXiv. 2025. arXiv:2508.00083. doi: 10.48550/arXiv.2508.00083
13. Nejjar M, Luca Z, Stiehle F, Weber I. LLMs for science: usage for code generation and data analysis. arXiv. 2023. arXiv:2311.16733. doi: 10.48550/arXiv.2311.16733
14. Joel S, Wu JJ, Fard FH. A survey on LLM-based code generation for low-resource and domain-specific programming languages. arXiv. 2024. arXiv:2410.03981. doi: 10.48550/arXiv.2410.03981
15. Wang X, Chen Y, Yuan L, Zhang Y, Li Y, Peng H, et al. Executable code actions elicit better LLM agents. Proceedings of the Forty-first International Conference on Machine Learning; 21–27 July 2024; Vienna, Austria.
16. Liu J, Xia CS, Wang Y, Zhang L. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. Proceedings of the Advances in Neural Information Processing Systems (NeurIPS 2023); 10–16 December 2023; New Orleans, LA, USA. [accessed on 27 September 2025]. Available from: https://proceedings.neurips.cc/paper_files/paper/2023/file/43e9d647ccd3e4b7b5baab53f0368686-Paper-Conference.pdf

17. Kazemitabaar M, Hou X, Henley A, Ericson BJ, Weintrop D, Grossman T. How novices use LLM-based code generators to solve CS1 coding tasks in a self-paced learning environment. Proceedings of the 23rd Koli Calling International Conference on Computing Education Research; 13–18 November 2023; Koli, Finland. p. 1–12. doi: 10.1145/3631802.3631806

18. Iverson KE. A programming language. New York (NY): John Wiley and Sons; 1962.

19. Falkoff AD, Iverson KE, Sussenguth EH. A formal description of system/360. IBM Syst J. 1964;3(2–3):198–261. doi: 10.1147/sj.32.0198

20. Falkoff AD, Iverson KE. The APL\360 terminal system. Proceedings of the Symposium on Interactive Systems for Experimental Applied Mathematics: Proceedings of the Association for Computing Machinery Inc. Symposium; 1 August 1967; Washington, DC, USA. p. 22–37.

21. Tavera ML, Alfonseca M, Rojas J. The IBM personal computer APL system. APL Quote Quad. 1984;14(4):333–7. doi: 10.1145/384283.801117

22. IBM Corporation. APL2 programming: language reference. No. SH20-9227. White Plains (NY): IBM Corporation; 1987.

23. F-Chart Software. EES: engineering equation solver for microsoft windows operating systems. Commercial and professional versions. Madison (WI): F-Chart Software; 2018 [accessed on 27 September 2025]. Available from: https://www.fchart.com/assets/downloads/ees_manual.pdf

24. F-Chart Software. Component library. [accessed on 27 September 2025]. Available from: https://fchartsoftware.com/ees/eeshelp/hs888.htm

25. Klein SA, Nellis GF. Mastering EES. Madison (WI): F-Chart Software; 2022.

26. Novak K. Numerical methods for scientific computing. the definitive manual for math geeks. 2nd ed. Sharon (VT): Equal Share Press; 2022 [accessed on 27 September 2025]. Available from: https://www.equalsharepress.com/media/NMFSC.pdf

27. Lecture #19: depth first search and strong components. 15-451/651: design & analysis of algorithms. Carnegie Mellon University, Pittsburgh Penn; 2018 [accessed on 27 September 2025]. Available from: https://www.cs.cmu.edu/ 15451-f18/lectures/lec19-DFS-strong-components.pdf

28. Çengel YA, Cimbala JM. Fluid mechanics. In: Fundamentals and applications. 4th ed. New York (NY): McGraw-Hill Global Education Holdings LLC.; 2018.

29. Klein SA, Nellis GF. Heat transfer. New York (NY): Cambridge University Press; 2009.

30. Alraba'nah Y, Sleit A, Qaddara I, Hiari M. Code generation by large language models: a comparative analysis of ChatGPT, Claude, and DeepSeek. Int J Electr Electron Eng Telecommun. 2026;15(1):19–28. doi: 10.18178/ijeetc.15.1.19-28

31. Guttag JV. Introduction to computation and programming using Python: with application to computational modeling and understanding data. 3rd ed. Cambridge (MA): The MIT Press; 2021.

32. Alfonseca M, Pulido E, Orosco R, de Lara J. OOCSMP: an object-oriented simulation language. Proceedings of the 9th European Simulation Symposium ESS97, Proceedings; 19–22 October 1997; Passau, Germany. p. 44–8.